Alexander Swartz – as7629

Dr. Kaoutar El Maghraoui

High Performance Machine Learning

18 February 2026

Assignment 1 Results

C1

N: 1000000 <T>: 0.001 sec B: 8.550 GB/sec F: 2.137 GFLOP/sec

N: 300000000 <T>: 0.287 sec B: 8.377 GB/sec F: 2.094 GFLOP/sec

C2

N: 1000000 <T>: 0.000 sec B: 29.655 GB/sec F: 29.655 GFLOP/sec

N: 300000000 <T>: 0.096 sec B: 25.007 GB/sec F: 25.007 GFLOP/sec

C3

N: 1000000 <T>: 0.000 sec B: 192.079 GB/sec F: 48.020 GFLOP/sec

N: 300000000 <T>: 0.033 sec B: 71.991 GB/sec F: 17.998 GFLOP/sec

C4

N: 1000000 <T>: 0.181 sec B: 0.044 GB/sec F: GFLOP/sec: 0.011

N: 300000000 <T>: 54.080 sec B: 0.044 GB/sec F: GLOP/sec: 0.011

C5

N: 1000000 <T>: 0.000 sec B: 74.874 GB/sec F: 18.718 GFLOP/sec

N: 300000000 <T>: 0.078 sec B: 30.780 GB/sec F: 7.695 GFLOP/sec

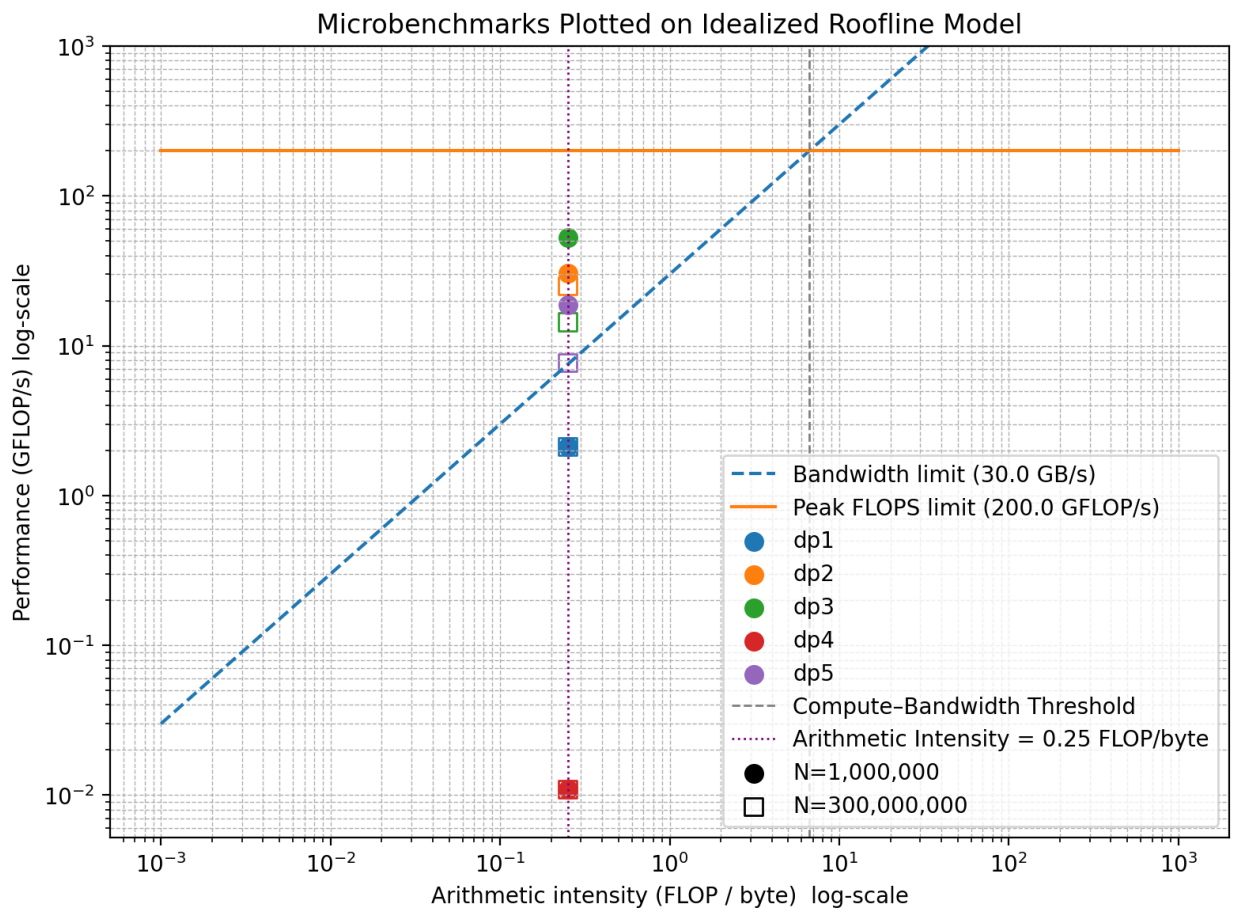
Q1

The rationale is to account for warm-up effects, such as when the vectors are loaded from RAM before moving to cache in the first measurements, whereas later measurements can access them from cache. The CPU also learns patterns within the loop branches and caches low-level instructions. Calls to libraries such as numpy.dot can take time to initialize on the first calls. The expected consequence is that measuring the second half of the iterations more accurately

measures the different microbenchmarks and their optimizations at their full potential when they've already passed these slower, earlier iterations.

The arithmetic mean is appropriate for finding average execution time because time is a raw value that has physical meaning and each iteration contributes equally to the final result (in the form of the denominator). Bandwidth and FLOPS were derived from (total execution time) / (total memory referenced in bytes) and (total execution time) / (total FLOPs), respectively. This is a simpler and faster way to compute averages for these rates than individually timing each measurement and using the harmonic mean, which would achieve the same result but slower and with more storage.

Q2



All 10 configurations were measured to have fewer GFLOPS than the assumed peak compute performance of 200GB/s, meaning they are all memory bound. This is shown in the roofline model by every point being below the horizontal "Peak FLOPS limit", and to the left of

the vertical “Compute-Bandwidth Threshold”, which measures what an idealized algorithm would accomplish if it had sufficient arithmetic intensity to reach the peak compute performance. Given the peak memory bandwidth of 30 GB/s and a peak performance of 200 GFLOP/s, an idealized algorithm would have required an arithmetic intensity of 200 / 30 = 6.667 FLOP/byte. Since the benchmarked algorithms had an arithmetic intensity of 0.25 FLOP/byte, they do not maximize the CPU’s compute capability and instead are memory bound.

The underlying reasons that microbenchmarks differ are from the different optimizations performed by the C compiler, python interpreter and chosen libraries. C implementations benefit from fixed types, contiguous layouts for the vectors, and compiler optimizations like loop unrolling. The difference in performance between Python microbenchmarks comes from the python overhead and lack of tuned libraries. The microbenchmarks that utilize more optimizations such as loop unrolling, parallelism, and SIMD all achieve higher GFLOPS with a smaller N. This is likely because larger vectors could not entirely fit in the caches, requiring more loading from DRAM which is slower. The aforementioned optimizations don’t necessarily scale when a larger N causes memory bounding to take over, which could have contributed to dp3 being fastest for N=100000000 but not N=300000000.

Dp4 significantly underperformed, with N=300000000 taking 54.080 seconds compared to the second longest of .280 seconds for dp1. This was likely due to Python interpreter overhead and lack of compiler optimizations. Instead of running compiled C like with np.dot, dp4 runs entirely in Python, incurring overhead from loop indexing and lack of parallelization, requiring each multiplication to happen sequentially.

Q3

For N=300000000, dp2 performed best with 25.007 GFLOPS, closely followed by dp3 (14.407 GLOPS) and dp5 (7.695 GFLOPS). Significantly worse were dp1 (2.121 GFLOPS) and dp4 (.011 GFLOPS). The microbenchmarks in C generally performed better than Python (especially dp4) because they get compiled to machine code, which can be executed much faster because of fixed types, known memory layouts, and aggressive optimizations taken by the compiler. These optimizations from using gcc -O3 include inlining functions, unrolling loops, and SIMD instructions. These improvements made dp1 –implemented in C – faster than dp4. Dp2 performed even better because of less loop overhead due to incrementing by 4 instead of 1.

More multiplications per loop can improve pipelining by letting the CPU overlap operations better and spending less time waiting to load from memory. Dp3 also performs well because it uses cblas_sdot, a highly tuned function with system-level optimizations to increase parallelism. Some of these optimizations include better SIMD and microarchitecture-specific tunings, where MKL chooses parameters for its optimizations based on hardware. Lastly, dp5 performed much better than the naïve Python solution in dp4 because it uses np.dot(), which allows all computation to be done in an optimized C implementation. This entirely avoids the Python bytecode that occurs with dp4's implementation. Np.dot() also uses SIMD and arranges memory to maximize bandwidth by reducing loads and stores.

Q4

The dot product results for all microbenchmarks with N=1000000 were 1000000, matching the analytic result. For N=30000000, however, the results were sometimes limited by the maximum values float32 can store, along with how much is being added to the result each iteration. The maximum consecutive integer a float32 can represent (when accumulating like in dp1) is 16777216. Starting here, the next representable integer is 16777218 due to the way floating point values sacrifice precision for great magnitude values. Therefore, once dp1and dp4 reach this number, a single loop will only ever try to add 1 to the result, which will keep being rounded down to 16777216. Similarly, dp2's result is 67108864, which is 4 * dp1's result because the loop unrolling with 4 indices at a time allow for 4 different lanes to reach 16777216 before being accumulated. Dp3 and dp4 result in the correct value because the dot product implementations from cblas and numpy don't implement the dot product as a running sum like the other implementations. Instead, they parallelize the multiplications in blocks and reduce them with higher precision to avoid the aforementioned rounding problems.